

Exercise 3: CASE Statements

SQL Fundamentals

Question 1

01

The products Table

A short product catalogue. Use the price column to bucket each item into a price tier.

▶ **TABLE** products

product_id	product_name	price
1	Laptop	1200.00
2	Phone	800.00
3	Keyboard	45.00
4	Monitor	300.00
5	Mouse	25.00

01

Classify each product by price into the three tiers below.

- 'Expensive' if price > 1000
- 'Mid-range' if price between 100 and 1000
- 'Budget' if price < 100

EXPECTED COLUMNS:

product_name

price

price_category

SQL code:

Last edit was now

▶ Run all (1000) ✓ Just now (5s) workspace default

```
--Question 1: Classify each product by price into the three tiers below.
SELECT product_name,
       price,
       CASE
         WHEN price > 1000 THEN 'Expensive'
         WHEN price BETWEEN 100 AND 1000 THEN 'Mid-range'
         WHEN price < 100 THEN 'Budget'
       END AS price_category
FROM brightlearn.master_data.products;
```

Add parameter

Table +

	product_name	price	price_category
1	Laptop	1200	Expensive
2	Phone	800	Mid-range
3	Keyboard	45	Budget
4	Monitor	300	Mid-range
5	Mouse	25	Budget

Question 2

02

The orders Table

Customer orders with the amount paid. Label each order by its monetary tier.

► TABLE orders

order_id	customer_name	amount
1	Alice	150.00
2	Bob	560.00
3	Charlie	999.99
4	Diana	45.50
5	Ethan	1200.00

02

Label each order by its value.

- 'High Value' for orders ≥ 1000
- 'Medium Value' for 500-999.99
- 'Low Value' for orders < 500

EXPECTED COLUMNS:

customer_name

amount

order_value_category

SQL code:

► Run all (1000) ▼



Just now (<1s)



workspace. default ▼

```
11  ---Question 2
12  ▼ SELECT customer_name,
13         amount,
14  ▼     CASE
15         WHEN amount >= 1000 THEN 'High Value'
16         WHEN amount BETWEEN 500 AND 999.99 THEN 'Medium Value'
17         WHEN amount < 500 THEN 'Low Value'
18         END AS order_value_category
19  FROM brightlearn.master_data.orders;
```

Add parameter

Table ▼



	customer_name	amount	order_value_category
1	Alice	150.00	Low Value
2	Bob	560.00	Medium Value
3	Charlie	999.99	Medium Value
4	Diana	45.50	Low Value
5	Ethan	1200.00	High Value

Question 3

03

The employees Table

Combine the department and salary columns to label each employee with a seniority tier.

▶ **TABLE** employees

emp_id	emp_name	department	salary
1	John	IT	85000
2	Sara	HR	60000
3	Mark	IT	75000
4	Lucy	Finance	95000
5	Tom	HR	55000

03

Categorize each employee's position level using both department and salary.

- If in 'IT' and salary > 80000 → 'Senior IT'
- If in 'HR' and salary > 55000 → 'Experienced HR'
- Otherwise → 'Staff'

EXPECTED COLUMNS:

emp_name

department

salary

position_level

SQL code:

▶ Run all (1000) ✓ 1 minute ago (<1s) workspace default

```
--Question 3
21 SELECT emp_name,
22         department,
23         salary,
24         CASE
25             WHEN department = 'IT' and salary>80000 THEN 'Senior IT'
26             WHEN department = 'HR' and salary>55000 THEN 'Experienced HR'
27             ELSE 'Staff'
28         END AS position_level
29 FROM brightlearn.master_data.employees_3;
```

Add parameter

Table +

Q F

	A ^B _C emp_name	A ^B _C department	1 ² ₃ salary	A ^B _C position_level
1	John	IT	85000	Senior IT
2	Sara	HR	60000	Experienced HR
3	Mark	IT	75000	Staff
4	Lucy	Finance	95000	Staff
5	Tom	HR	55000	Staff

Question 4

04

The students Table

A list of student scores. Convert numeric scores into letter grades.

► TABLE students

student_id	student_name	score
1	Anna	92
2	Ben	76
3	Cara	59
4	David	83
5	Ella	68

04

Assign each student a letter grade based on their score.

- $\geq 90 \rightarrow$ 'A'
- 80-89 $\rightarrow$ 'B'
- 70-79 $\rightarrow$ 'C'
- 60-69 $\rightarrow$ 'D'
- $< 60 \rightarrow$ 'F'

EXPECTED COLUMNS:

student_name

score

grade

SQL code:

▶ Run all (1000) ✓ 1 minute ago (1s) workspace . default ▼

```
32 SELECT student_name,  
33        score,  
34        CASE  
35            WHEN score >= 90 THEN 'A'  
36            WHEN score BETWEEN 80 AND 89 THEN 'B'  
37            WHEN score BETWEEN 70 AND 79 THEN 'C'  
38            WHEN score BETWEEN 60 AND 69 THEN 'D'  
39            WHEN score < 60 THEN 'F'  
40        END AS grade  
41 FROM brightlearn.master_data.students_2;
```

Add parameter

Table ▼ +

	^A _C student_name	¹ ₂ ³ score	^A _C grade
1	Anna	92	A
2	Ben	76	C
3	Cara	59	F
4	David	83	B
5	Ella	68	D

Question 5

05

The deliveries Table

Delivery times in minutes. Label each delivery by its performance.

▶ TABLE deliveries

delivery_id	delivery_time_minutes
1	45
2	80
3	30
4	65
5	100

05

Label delivery performance based on the time taken.

- ≤ 30 mins → 'Fast'
- 31-60 mins → 'On Time'
- > 60 mins → 'Late'

EXPECTED COLUMNS:

delivery_id

delivery_time_minutes

performance

SQL code:

▶ Run all (1000) ✓ Just now (1s) workspace default

```
43 --Question 5
44 SELECT delivery_id,
45         delivery_time_minutes,
46 CASE
47     WHEN delivery_time_minutes <= 30 THEN 'Fast'
48     WHEN delivery_time_minutes BETWEEN 31 AND 60 THEN 'On Time'
49     WHEN delivery_time_minutes > 60 THEN 'Late'
50 END AS performance
51 FROM brightlearn.master_data.deliveries;
```

Add parameter

Table +

Q Y

	1 ² ₃ delivery_id	1 ² ₃ delivery_time_minutes	A ^B _C performance
1	1	45	On Time
2	2	80	Late
3	3	30	Fast
4	4	65	Late
5	5	100	Late

Question 6

06

The tickets Table

Support tickets stored with a numeric priority. Convert the priority into a readable label.

▶ TABLE tickets

ticket_id	issue_type	priority
1	Login issue	1
2	Server down	3
3	Slow system	2
4	Email error	2
5	Password reset	1

06

Convert the numeric priority into a readable label.

- 3 → 'High'
- 2 → 'Medium'
- 1 → 'Low'

EXPECTED COLUMNS:

issue_type

priority

priority_label

SQL code:

▶ Run all (1000) ✓ Just now (1s) workspace default

```
53  --Question 6
54  SELECT issue_type,
55         priority,
56         CASE
57             WHEN priority = 3 THEN 'High'
58             WHEN priority = 2 THEN 'Medium'
59             WHEN priority = 1 THEN 'Low'
60         END AS priority_label
61  FROM brightlearn.master_data.tickets;
```

Add parameter

Table +

	^A _C issue_type	¹ ₃ priority	^A _C priority_label
1	Login issue	1	Low
2	Server down	3	High
3	Slow system	2	Medium
4	Email error	2	Medium
5	Password reset	1	Low

Question 7

07

The attendance Table

Calculate each student's attendance percentage and classify the result.

► TABLE attendance

student_id	days_present	total_days
1	45	50
2	30	50
3	48	50
4	25	50
5	50	50

07

Calculate attendance percentage and classify the result.

- $\geq 90\%$ → 'Excellent'
- 75-89% → 'Good'
- $< 75\%$ → 'Needs Improvement'

EXPECTED COLUMNS:

student_id

attendance_percentage

attendance_status

SQL code:

```

63  --Question 7
64  SELECT
65      student_id,
66      (days_present/total_days)*100 AS attendance_percentage,
67      CASE
68          WHEN attendance_percentage>=90 THEN 'Excellent'
69          WHEN attendance_percentage BETWEEN 75 AND 89 THEN 'Good'
70          WHEN attendance_percentage<75 THEN 'Needs Improvement'
71      END AS attendance_status
72  FROM brightlearn.master_data.attendance:

```

Add parameter

Table ▾ +

	1 ² ₃ student_id	1.2 attendance_percentage	A ^B _C attendance_status
1	1	90	Excellent
2	2	60	Needs Improvement
3	3	96	Excellent
4	4	50	Needs Improvement
5	5	100	Excellent

Question 8

08 The products_inventory Table

Stock levels per product. Label each row with the correct stock status.

► TABLE products_inventory

product_id	stock_qty
1	5
2	0
3	25
4	10
5	3

08 Label the stock status of each product.

- 0 → 'Out of Stock'
- 1-5 → 'Low Stock'
- > 5 → 'In Stock'

EXPECTED COLUMNS:

product_id

stock_qty

stock_status

SQL code:

► Run all (1000) ✓ Just now (1s) workspace default

```
81 --Question 8
82 SELECT product_id,
83         stock_qty,
84         CASE
85             WHEN stock_qty = 0 THEN 'Out of stock'
86             WHEN stock_qty BETWEEN 1 AND 5 THEN 'Low Stock'
87             WHEN stock_qty > 5 THEN 'In Stock'
88         END AS stock_status
89 FROM brightlearn.master_data.products_inventory;
```

Add parameter

Table +

	1 ² ₃ product_id	1 ² ₃ stock_qty	A ^B _C stock_status
1	1	5	Low Stock
2	2	0	Out of stock
3	3	25	In Stock
4	4	10	In Stock
5	5	3	Low Stock

Question 9

09

The classes Table

Classes with their enrolled student counts. Classify each class by size.

► TABLE classes

class_id	subject	enrolled_students
1	Math	30
2	English	25
3	Science	15
4	Art	5
5	History	20

09

Classify each class by the number of enrolled students.

- $\geq 25 \rightarrow$ 'Large'
- $10-24 \rightarrow$ 'Medium'
- $< 10 \rightarrow$ 'Small'

EXPECTED COLUMNS:

subject

enrolled_students

class_size_category

SQL code:

► Run all (1000) ▼

✓ Just now (1s) workspace default ▼

```
91  --Question 9
92  SELECT subject,
93         enrolled_students,
94         CASE
95             WHEN enrolled_students >= 25 THEN 'Large'
96             WHEN enrolled_students BETWEEN 10 AND 24 THEN 'Medium'
97             WHEN enrolled_students < 10 THEN 'Small'
98         END AS class_size_category
99  FROM brightlearn.master_data.classes;
```

Add parameter

Table ▼



	^A _C subject	¹ ₃ enrolled_students	^A _C class_size_category
1	Math	30	Large
2	English	25	Large
3	Science	15	Medium
4	Art	5	Small
5	History	20	Medium

Question 10

10

The payments Table

Combine the payment method and amount to decide whether a discount applies.

► TABLE payments

payment_id	amount	payment_method
1	50.00	Card
2	200.00	Cash
3	150.00	Card
4	75.00	PayPal
5	300.00	Cash

10

Apply a discount flag based on the payment method and amount.

- If `payment_method = 'cash'` and `amount ≥ 200` → `'Eligible for Discount'`
- Otherwise → `'Not Eligible'`

EXPECTED COLUMNS:

payment_id

payment_method

amount

discount_eligibility

SQL code:

► Run all (1000) ✓ 11:58 AM (7s) workspace default

```

100
101 --Question 10
102 SELECT payment_id,
103         payment_method,
104         amount,
105         CASE
106             WHEN payment_method = 'cash' AND amount >= 200 THEN 'Eligible for Discount'
107             ELSE 'Not Eligible'
108         END AS discount_eligible
109 FROM brightlearn.master_data.payments;

```

Add parameter

Table	+	Q	Y	I	O	E	^
	1 ² 3 payment_id	A ^B C payment_method	1 ² 3 amount	A ^B C discount_eligible			
1	1	Card	50	Not Eligible			
2	2	Cash	200	Not Eligible			
3	3	Card	150	Not Eligible			
4	4	PayPal	75	Not Eligible			
5	5	Cash	300	Not Eligible			